



# Working With APIs in React



REACT

18+



JAVASCRIPT

ES6+



API

INTEGRATION



VITE

LATEST

Created by: Abhay Tiwari|[LinkedIn](https://www.linkedin.com/in/abhay-tiwari-satna/):  
<https://www.linkedin.com/in/abhay-tiwari-satna/>



## Introduction

In modern React development, interacting with APIs is an essential skill — it allows your app to fetch real-time data, display dynamic content, and handle asynchronous operations efficiently.

In this lesson, you'll learn how to fetch and manage data in React using both `.then()` and `async/await` approaches, handle loading and error states, and dynamically display API data using React components and hooks.

We'll build a mini Crypto Dashboard powered by the [CoinGecko API](#), where you'll fetch and render live cryptocurrency data, apply filters, sorting, and pagination — all in a professional, real-world structure.



## What You'll Learn

By the end of this guide, you'll confidently understand and be able to:

1. ⚡ Use **useEffect** for performing side effects (like API calls).
2. 🌐 Fetch data from a public API using Fetch API and `async/await`.
3. 🔄 Manage loading, success, and error states effectively.
4. 🧱 Structure your app into reusable components like `CoinCard`, `LimitSelector`, `FilterInput`, etc.
5. ⚙️ Work with Environment Variables (**.env** files) safely in Vite + React.
6. 🔍 Implement search filters, sorting, and limit selectors dynamically.
7. 💡 Understand why **useEffect** runs twice in development and how React Strict Mode works.
8. 🎨 Build a clean, interactive, and data-driven React interface from scratch.



## Key Concepts Covered

| Category              | Concept                                         | Description                                                                |
|-----------------------|-------------------------------------------------|----------------------------------------------------------------------------|
| React Hooks           | <code>useEffect</code>                          | Run side effects (like fetching data) after rendering                      |
|                       | <code>useState</code>                           | Manage dynamic values like data, loading, errors                           |
| API Fetching          | Fetch API                                       | Use <code>.then()</code> or <code>async/await</code> to make HTTP requests |
|                       | Error Handling                                  | Manage network failures or invalid responses gracefully                    |
| Data Flow             | Component Composition                           | Split large components into smaller, reusable ones                         |
|                       | Props                                           | Pass data from parent to child components cleanly                          |
| State Management      | Conditional Rendering                           | Show loading/error states or data conditionally                            |
| Environment Variables | <code>.env</code> and <code>VITE_</code> prefix | Securely store and access config values                                    |
| User Interaction      | Filtering                                       | Search coins by name or symbol                                             |
|                       | Sorting                                         | Sort data by price, market cap, or change                                  |
|                       | Limit Control                                   | Adjust number of displayed items dynamically                               |
| UI/UX Enhancements    | Dynamic Styling                                 | Apply conditional classes for positive/negative changes                    |
|                       | Data Formatting                                 | Use <code>.toLocaleString()</code> for currency display                    |
| React Behavior        | Strict Mode                                     | Understand why effects run twice in development                            |

## Understanding the Context

We've already learned about side effects and how to use the `useEffect` hook. One of the most common use cases for this hook is making HTTP requests.

When working with client-side React (not using frameworks like Next.js), data fetching typically happens inside:



- Event handlers, or
- Lifecycle methods like `useEffect`

There are also more advanced tools for data fetching:

- React Router Loaders
- TanStack Query (formerly React Query)
- Server Components in Next.js

However, before learning those, it's essential to master the basics. So, in this note, we'll fetch data the traditional way using `useEffect`.

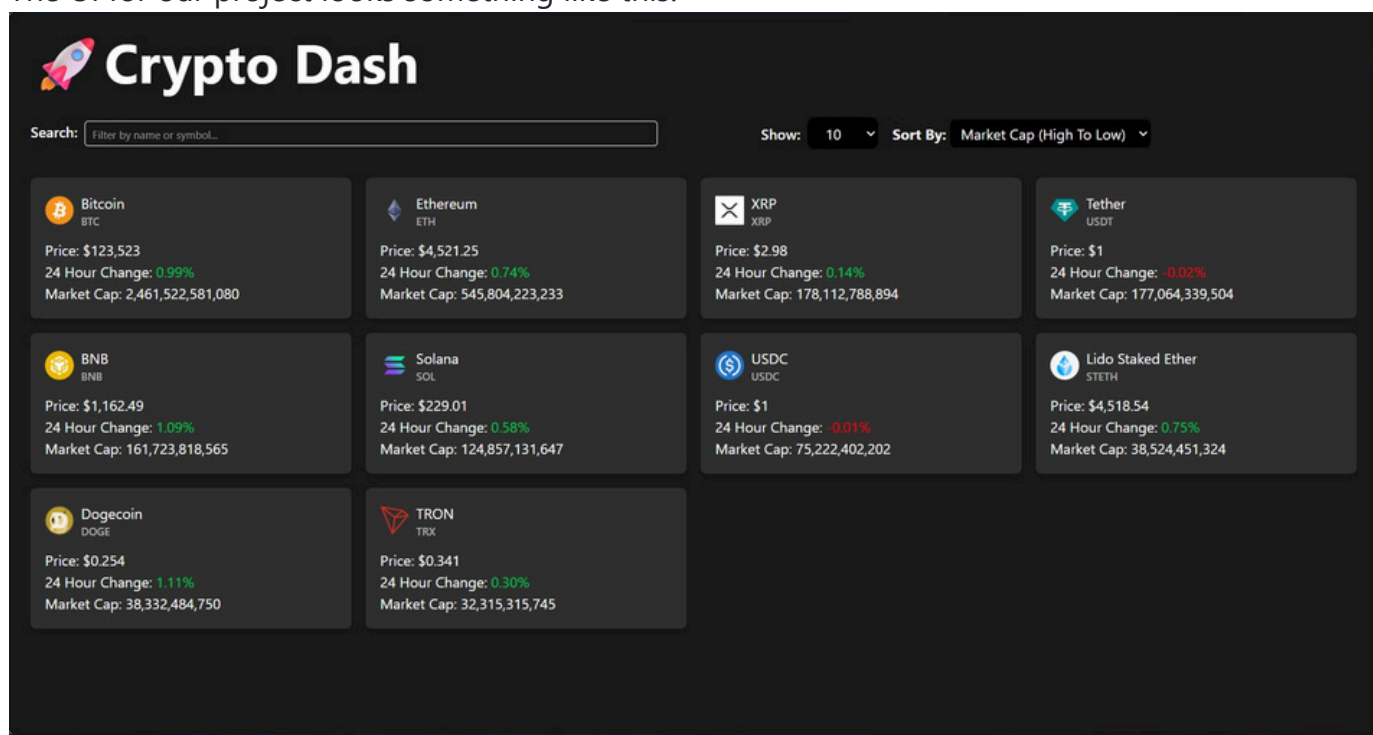
We'll build a Cryptocurrency Dashboard using the [CoinGecko API](#), handle loading & error states, and display live crypto data beautifully.

## API Info & Project Setup

we'll be working with the `coins/markets` endpoint from `goingecko` APIs. This lets us query all the supported coins with price, market cap, volume and market related data. That's what we need for our project.

### Project:

1. Make a folder for the project and do the basics such as installing react and tailwind or you may proceed with vanilla css.
2. The UI for our project looks something like this:



- Build the UI like this or whatever we want now we will be learning how we can build this by fetching the data.

## Making HTTP Requests

In this section, we will learn how to make HTTP requests in React. Usually, we can do this either from an event handler or using the `useEffect` hook if you want the data to be fetched when the component is mounted.

Let's start by opening the `App.jsx` file and importing both `useEffect` and `useState` from React and adding a piece of state for the

1. coins
2. the loading state and
3. state for any errors if there are any.

The loading state will be used to show a loading spinner while the data is being fetched.

```
import { useEffect, useState } from "react";

const App = () => {
  const [coins, setCoins] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  return (
    <div>
      <h1>🚀 Crypto Dash</h1>
    </div>
  );
};

export default App;
```

You can notice that I had set the loading state to true by default. This is because we want to show a loading spinner while the data is being fetched. This is a common practice to start with it as true and then set it to false once the data is fetched. We also initialized the coins state to an empty array. This is where we will store the data fetched from the API. We also set the error state to an empty array. We will use this to store any errors that occur during the fetch.

## Making The Request

Now, let's add the `useEffect` hook to fetch the data from the CoinGecko API. This is a free API that provides a lot of data about cryptocurrencies. We will use the `/coins/markets` endpoint to fetch the data. You can find more information about the API [here](#).



We will also add some query params to the request. We will set the `vs_currency` to `usd` and the order to `market_cap_desc`. This will return the top 10 cryptocurrencies by market cap in descending order.

I am also going to include `sparkline=false`. Sparkline is a coins price trend over a time period. We won't be using that data so I'm setting it to `false` to lighten the data load a bit.

Here is the URL:

```
https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_page=
```

In fact, we can store it in a variable for now. Ultimately, we will move it to a `.env` file.

```
import { useEffect, useState } from "react";
const API_URL =
  "https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_pa

const App = () => {
  // ...
};
```

## Fetching data using `.then` syntax

We will use the `fetch` function to make the request. I am initially going to show you how to use the `.then()` syntax and then I will refactor to `async/await`, because there are a few gotchas.

Add the following code inside the `useEffect` hook:

```
const App = () => {
  const [coins, setCoins] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState([]);

  useEffect(() => {
    fetch(API_URL)
      .then((res) => {
        if (!res.ok) {
          throw new Error("Failed to fetch data");
        }
        return res.json();
      })
      .then((data) => {
        console.log(data);
        setCoins(data);
        setLoading(false);
      })
  });
};
```



```

    })
    .catch((err) => {
      setError(err.message);
      setLoading(false);
    });
  }, []);

  return (
    <div>
      <h1>🚀 Crypto Dash</h1>
    </div>
  );
};

export default App;

```

We called the fetch function with the API\_URL and then we chained a .then() method to handle the response. If the response is not ok, we throw an error. If it is ok, we parse the JSON and set the coins state to the data. We also set the loading state to false to hide the loading spinner. If there is an error, we set the error state to the error message and set the loading state to false.

We are not displaying the data yet, but if you open the console, you should see the data being logged. Also, check the React Devtools 'Components' tab in the browser to see the state being updated.

Some of the data that we get back that we will be using is:

- name
- image
- current\_price
- market\_cap
- price\_change\_percentage\_24h

If you add something to the url to make it invalid, you should see the error message get added to the state.

## ⚠️ Why Is My useEffect Running Twice?

You may notice that in the console your data is being displayed twice. This is because of how Strict Mode works in development. It actually intentionally runs some parts of your code twice — especially things like useEffect — to help catch bugs early.

Here's what's happening:

- When your component mounts, React runs the effect, like your fetch() call.



- Then, React unmounts the component, as a test.
- Then it mounts it again and runs the effect a second time.

This doesn't happen in production — it's only in development to make sure things like API calls, timers, and subscriptions are set up and cleaned up properly. So if you see your data printed twice or a fetch call running more than once, don't panic — it's just React being cautious.

If you're curious, this is all controlled by `<React.StrictMode>` in your app's entry file. You can remove it to stop the double run during testing, but it's helpful to leave it in while developing to catch potential bugs.

## ⚡ Fetching data Using Async/Await in `useEffect`

There is a small gotcha when learning the `async/await` syntax. You can't use `await` directly in the `useEffect` hook. You need to create a separate function and call it inside the `useEffect` hook.

✗ For instance, you can NOT do this:

```
useEffect(async () => {
  const response = await fetch(API_URL);
  //...
}, []);
```

✓ Instead, you need to create a separate function and call it inside the `useEffect` hook:

```
useEffect(() => {
  const fetchCoins = async () => {
    try {
      const res = await fetch(API_URL);
      if (!res.ok) throw new Error('Failed to fetch data');
      const data = await res.json();
      setCoins(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  fetchCoins();
}, []);
```

In the code above, we created a function called `fetchCoins` that is `async`. We then called the function inside the `useEffect` hook. We used `try/catch` to handle any errors that might occur. We

also used the `finally` block to set the `loading` state to `false`. This way, we don't have to call `setLoading(false)` in both the `try` and `catch` blocks.

## Display Coin Data

---

Now that we have the data from the API, we can output it in the JSX. Remember, we have to account for both the loading and error states.

We could do this a few ways. We can have separate if statements for each state, or we can use a ternary operator.

So we could do something like this:

```
if (loading) {
  return <p>Loading...</p>;
}

if (error) {
  return (
    <div className='error'>
      <p>❌ {error}</p>
    </div>
  );
}

return (
  <div>
    <h1>🚀 Crypto Dash</h1>
  </div>
);
```

I prefer to do it in the same return as a short-circuit condition:

```
return (
  <div>
    <h1>🚀 Crypto Dash</h1>
    {loading && <p>Loading...</p>}
    {error && (
      <div className='error'>
        <p>❌ {error}</p>
      </div>
    )}
  </div>
);
```

If you reload, you should see the loading message real quick.





## Outputting the Data

Let's now add the data that is in the coins state. We also want to make sure there is no error and it's not loading before we try and output the data.

Here is the final code:

```
import { useEffect, useState } from 'react';
const API_URL =
  'https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_pa

const App = () => {
  const [coins, setCoins] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchCoins = async () => {
      try {
        const res = await fetch(API_URL);
        if (!res.ok) throw new Error('Failed to fetch data');
        const data = await res.json();
        setCoins(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchCoins();
  }, []);

  return (
    <div>
      <h1>🚀 Crypto Dash</h1>
      {loading && <p>Loading...</p>}
      {error && (
        <div className='error'>
          <p>❌ {error}</p>
        </div>
      )}

      {!loading&& !error && (
        <main className='grid'>
          {coins.map((coin) => (
            <div className='coin-card' key={coin.id}>
              <div className='coin-header'>
                <img src={coin.image} alt={coin.name} className='coin-image' />
              </div>
              <h2>{coin.name}</h2>
              <p className='symbol'>{coin.symbol.toUpperCase()}</p>
            </div>
          )}
        </main>
      )}
    </div>
  )}

```



```

        </div>
      </div>
      <p>Price: ${coin.current_price.toLocaleString()}</p>
      <p
        className={
          coin.price_change_percentage_24h >= 0
            ? 'positive'
            : 'negative'
        }
      >
        24h Change: {coin.price_change_percentage_24h.toFixed(2)}%
      </p>
      <p>Market Cap: ${coin.market_cap.toLocaleString()}</p>
    </div>
  )))
</main>
})
);</div>
};

```

```
export default App;
```

You should now see a nice layout of all the coins with their respective data. We used some formatting methods like `toLocaleString()` and `toFixed()` to make the numbers more readable. We also added a conditional class to the 24h change percentage to show green if it's positive and red if it's negative.

If you see an error, make sure you have the correct API URL and that you are not exceeding the rate limit.

## Coin Card

Now we can move the Card logic into a separate component. Create a new file at `src/components/CoinCard.js` in the `src` folder and add the following code:

```

const CoinCard = ({ coin }) => {
  return (
    <div className='coin-card' key={coin.id}>
      <div className='coin-header'>
        <img src={coin.image} alt={coin.name} className='coin-image' />
        <div>
          <h2>{coin.name}</h2>
          <p className='symbol'>{coin.symbol.toUpperCase()}</p>
        </div>
      </div>
      <p>Price: ${coin.current_price.toLocaleString()}</p>
      <p
        className={
          coin.price_change_percentage_24h >= 0 ? 'positive' : 'negative'
        }
      >

```



```

    >
    24h Change: {coin.price_change_percentage_24h.toFixed(2)}%
  </p>
  <p>Market Cap: ${coin.market_cap.toLocaleString()}</p>
</div>
);
};

```

export default CoinCard;

Bring it into the App.js file and use it:

```

import { useEffect, useState } from 'react';
import CoinCard from './components/CoinCard';

const API_URL =
  'https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_page=10';

const App = () => {
  const [coins, setCoins] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchCoins = async () => {
      try {
        const res = await fetch(API_URL);
        if (!res.ok) throw new Error('Failed to fetch data');
        const data = await res.json();
        setCoins(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchCoins();
  }, []);

  return (
    <div>
      <h1>🚀 Crypto Dash</h1>
      {loading && <p>Loading...</p> }
      {error && (
        <div className='error'>
          <p>❌ {error}</p>
        </div>
      )}

      {!loading && !error && (

```



```

    <main className='grid'>
      {coins.map((coin) => (
        <CoinCard coin={coin} key={coin.id} />
      ))}
    </div>
  )}
</div>
);
};

export default App;

```

## Environment Variables

Environment variables are special values stored outside your code that let you configure your app without hardcoding sensitive or environment-specific info. In a Vite + React project, using environment variables is straightforward and built into the tool.

### Defining Environment Variables with .env Files

Vite projects support dotenv files for defining environment variables. You typically place these files at the project root (where your vite.config.js or index.html lives). Vite will automatically load these files on startup without any extra packages. The naming of the files can target all environments or specific modes (development, production, etc.). Common file names include:

- .env – Loaded in all cases (base default variables)
- .env.local – Loaded in all cases, but intended to be local-only (not checked into git)
- .env.development, .env.production (i.e. .env.[mode]) – Loaded only for that specific mode (when running in that mode)
- .env.development.local, .env.production.local (i.e. .env.[mode].local) – Mode-specific and local-only (not in git)

When you run the dev server or build, Vite determines the "mode" (development by default for vite dev, and production by default for vite build). It will load .env and .env.local first, then the mode-specific files. Mode-specific env files override the generic ones if they define the same variable. For example, if both .env and .env.production define VITE\_API\_URL, the value from .env.production will be used in production builds.

### The VITE\_ Prefix Requirement

One important rule in Vite is that only variables prefixed with VITE\_ are exposed to your client-side code. This is a safety feature to prevent accidentally leaking sensitive keys. Vite will load all the variables from your .env files, but any variable that does not start with VITE\_ will be omitted from the app's front-end code

□



For example, suppose your .env contains:

```
VITE_SOME_KEY=123
DB_PASSWORD=foobar
```

In this case, VITE\_SOME\_KEY will be available in your React app, but DB\_PASSWORD will not be exposed. If you try to access it, it will be undefined in the browser. This behavior is by design – you might keep truly sensitive secrets (like database passwords) in .env files for backend or build processes, but you wouldn't want them accidentally shipped to the client.

Why the prefix? By enforcing a prefix, Vite makes you explicitly mark which env vars should be visible to front-end code. This convention is similar to how Create React App uses the REACTAPP prefix. In Vite's case, the prefix is VITE\_. (You can change this prefix via the envPrefix configuration, but in most cases the default is recommended.)

## Accessing Environment Variables in React Components

In a Vite + React app, you access env variables through the special `import.meta.env` object. This object includes all your `VITE_*` variables (as well as some built-in variables like mode flags). For example, using the variables from above:

```
export default function App() {
  const apiUrl = import.meta.env.VITE_API_URL;
  const featureFlag = import.meta.env.VITE_FEATURE_FLAG;

  console.log('API URL:', apiUrl);
  console.log('Feature enabled?', featureFlag);
}
```

If you have worked with environment variables in Node on the backend, you are probably used to using `process.env` to access them. Vite does not use `process.env` in the browser, as it is not available there. Instead, you use `import.meta.env` to access your environment variables in the front-end code.

## Security Considerations

As I mentioned, when using `VITE*` prefix, Vite will expose those variables to the client-side code. This means that anyone can see them in the browser's developer tools. Therefore, you should never put sensitive information (like API keys or passwords) in your .env files if they are prefixed with `VITE*`. Always keep sensitive information on the server side and use environment variables only for non-sensitive configuration values.

There are certain keys that you can make public. For example, Stripe has a public key and a secret key. The public key can be exposed to the client-side code, but the secret key should never be exposed.



Google Maps and other Google APIs are safe as long as you implement the restrictions from the dashboard. The Firebase client SDK keys can be public as well.

Anything that says "secret" or "private" should never be exposed to the client-side code. For this, you have a few options:

- Create a backend server that acts as a proxy for your API requests. This way, you can keep your API keys secret on the server side and only expose the necessary endpoints to the client.
- Serverless functions are another option. You can create serverless functions that act as a proxy for your API requests. You make a request to the serverless function, and it makes the request to the API with the secret key. You could use something like AWS Lambda, Netlify Functions, or Vercel Functions to create these serverless functions.
- Cloudflare workers are another option. You can create a Cloudflare worker that acts as a proxy for your API requests. You make a request to the Cloudflare worker, and it makes the request to the API with the secret key.
- Secrets Managers are another option. You can use a secrets manager to store your API keys and other sensitive information.

## Move Our API URL to an Environment Variable

Let's move our Coingecko API URL to an environment variable.

We will create a new file called .env in the root of our project and add the following line:

```
VITE_COINS_API_URL = "https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd"
```

I am not putting the query params except for the vs\_currency because that one will not change and is required.

In the App.jsx file, we will import the environment variable using import.meta.env:

```
const API_URL = import.meta.env.VITE_COINS_API_URL;
```

When we make the request is where we will add the rest of the params:

```
const res = await fetch(
  `${API_URL}&order=market_cap_desc&per_page=10&page=1&sparkline=false`
);
```

Now, we can use the environment variable in our code. This way, we can easily change the API URL without having to change the code. We can also use different API URLs for different environments (development, production, etc.) by creating different .env files.



## 🚫 .gitignore File

In many cases you will have sensitive info in these files and you wouldn't commit them. In our case, we do not. It is just the URL. So we do not have to use .env.local or add the .env file to the .gitignore file. But you still can if you want.

## 1 2 3 4 Limit Select

Right now, we are showing 10 coins, however we can get more than that. Let's add a select list to change the limit and reflect that in the UI by changing the params on the API call.

Add the following state to the App.jsx file:

```
const [limit, setLimit] = useState(10);
```

This will hold the limit value. The limit state will be used to store the value of the select input.

Next, we need to add a select input to the UI. Open the App.jsx file and add the following code below the heading:

```
<div className='controls'>
  <label htmlFor='limit'>Show:</label>
  <select
    id='limit'
    value={limit}
    onChange={(e) => setLimit(Number(e.target.value))}
  >
    <option value='5'>5</option>
    <option value='10'>10</option>
    <option value='20'>20</option>
    <option value='50'>50</option>
    <option value='100'>100</option>
  </select>
</div>
```

This will create a select input with the options to show 5, 10, 20, 50, and 100 coins. The onChange event will update the limit state with the selected value.

Now, we need to update the API call to use the limit state. Open the App.jsx file and update the API call to include the limit parameter. you also want this to run whenever the limit state changes. So, update the useEffect hook to include the limit state as a dependency:

```
useEffect(() => {
  const fetchCoins = async () => {
    try {
```



```

const res = await fetch(
  `${API_URL}&order=market_cap_desc&per_page=${limit}&page=1&sparkline=false`
);
if (!res.ok) throw new Error('Failed to fetch data');
const data = await res.json();
setCoins(data);
} catch (err) {
  setError(err.message);
} finally {
  setLoading(false);
}
};

fetchCoins();
}, [limit]);

```

## Move To It's Own Component

Let's move the limit selector to it's own component.

Create a new file in `src/components` called `LimitSelector.jsx`. In this file, we will create a functional component that will render the select input.

```

const LimitSelector = ({ limit, onLimitChange }) => {
  return (
    <div className='controls'>
      <label htmlFor='limit'>Show:</label>
      <select
        id='limit'
        value={limit}
        onChange={(e) => onLimitChange(Number(e.target.value))}>
        <option value='5'>5</option>
        <option value='10'>10</option>
        <option value='20'>20</option>
        <option value='50'>50</option>
        <option value='100'>100</option>
      </select>
    </div>
  );
};

export default LimitSelector;

```

Now, bring it into the `App.jsx` file and use it. You will need to import it at the top of the file:

```

import LimitSelector from './components/LimitSelector';

```



Replace the `<div className='controls'>` with the `LimitSelector` component:

```
<LimitSelector limit={limit} onLimitChange={setLimit} />
```

That's it! You have successfully moved the limit selector into its own component. Now, you can reuse this component in other parts of your app if needed.

## Filter Coins

At the moment, we are displaying a list of coins. We can change the limit but I want the user to be able to filter to find a certain coin.

Open the `App.jsx` file and add this new state value:

```
const [filter, setFilter] = useState('');
```

This will be used to filter the coins.

## Filter Component

Create a new file at `src/components/FilterInput.jsx` and add the following code:

```
const FilterInput = ({ filter, onFilterChange }) => {
  return (
    <div className='filter'>
      <input
        type='text'
        placeholder='Filter by name or symbol...'
        value={filter}
        onChange={(e) => onFilterChange(e.target.value)}
      />
    </div>
  );
};

export default FilterInput;
```

It takes in the `filter` value and a function to call when the input changes.

## Update App.jsx

Now in the `App.jsx` file, import the new component:

```
import FilterInput from './components/FilterInput';
```

Add the following function above the return statement:

```
const filteredCoins = coins.filter(
  (coin) =>
    coin.name.toLowerCase().includes(filter.toLowerCase()) ||
    coin.symbol.toLowerCase().includes(filter.toLowerCase())
);
```

This will filter the coins based on the name or symbol.

We will now embed the component. I want the input and the limit selector to be side by side, so let's add a wrapper element to the two components.

```
<div className='top-controls'>
  <FilterInput filter={filter} onFilterChange={setFilter} />
  <LimitSelector limit={limit} onLimitChange={setLimit} />
</div>
```

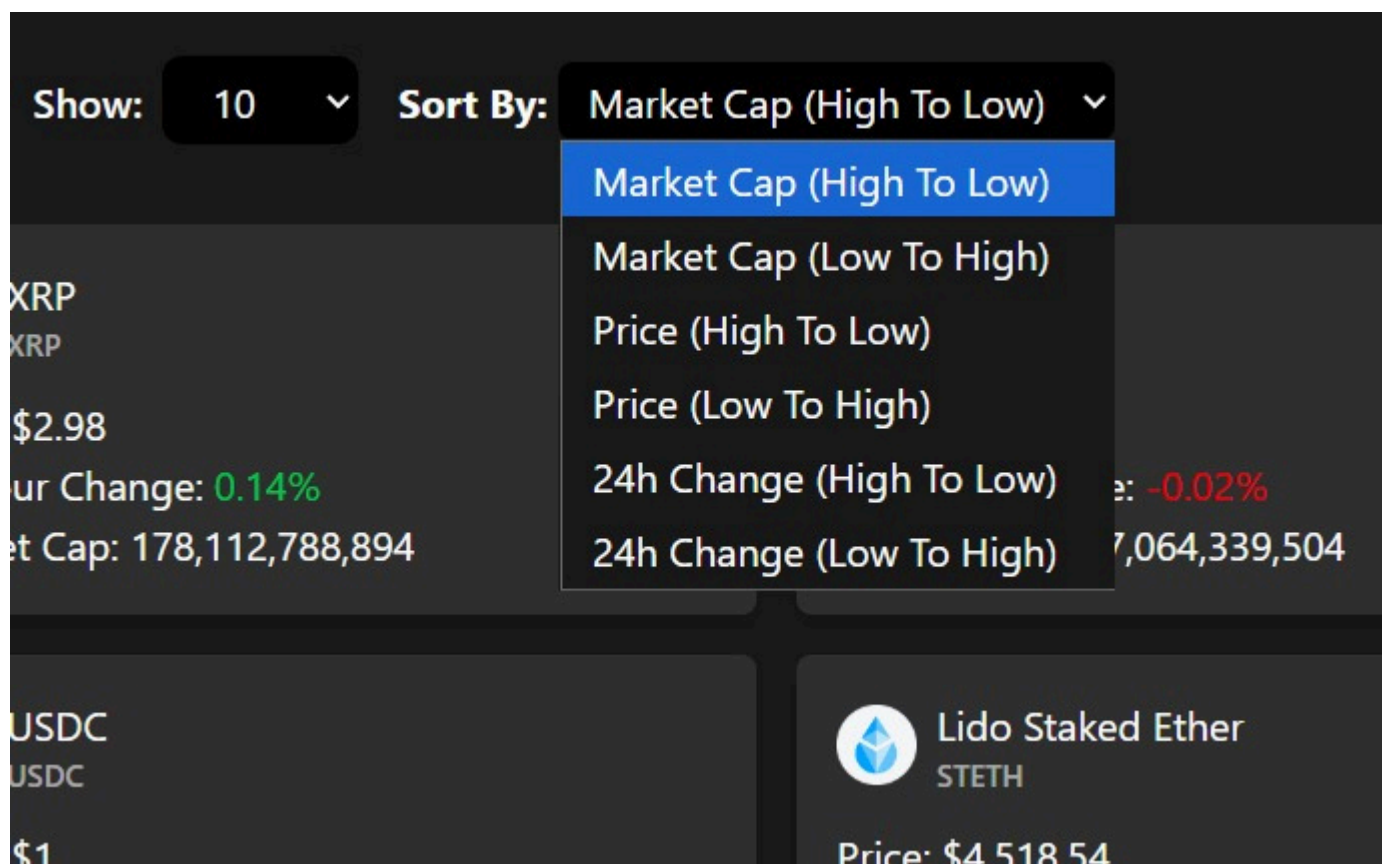
Now we need to make sure that if there is a filter input, we map over those instead of all coins:

```
{
  !loading && !error && (
    <div className='grid'>
      {filteredCoins.length > 0 ? (
        filteredCoins.map((coin) => <CoinCard coin={coin} key={coin.id} />)
      ) : (
        <p>No coins match your filter.</p>
      )}
    </div>
  );
}
```

Now you should be able to filter the coins by name or symbol. If you type in the input, it will filter the list of coins to only show those that match the input.

## Sort Selector

---



Let's add some sorting options for the prices and market cap.

Open the `src/App.jsx` file and add a new piece of state to hold the sorting option.

```
const [sortBy, setSortBy] = useState('market_cap_desc');
```

Create a new component at `src/components/SortSelector.jsx` to hold the sorting options.

```
const SortSelector = ({ sortBy, onSortChange }) => {
  return (
    <div className='controls'>
      <label htmlFor='sort'>Sort by:</label>
      <select
        id='sort'
        value={sortBy}
        onChange={(e) => onSortChange(e.target.value)}
      >
        <option value='market_cap_desc'>Market Cap (High to Low)</option>
        <option value='price_desc'>Price (High to Low)</option>
        <option value='price_asc'>Price (Low to High)</option>
        <option value='change_desc'>24h Change (High to Low)</option>
        <option value='change_asc'>24h Change (Low to High)</option>
      </select>
    </div>
  );
};
```

```
export default SortSelector;
```

This component will render a dropdown with the sorting options. The `onSortChange` prop will be called when the user selects a new sorting option.

Import the `SortSelector` component in `src/App.jsx`:

```
import SortSelector from './components/SortSelector';
```

Now add it to the return statement just below the `FilterInput` and `LimitSelector` components:

```
<div className='top-controls'>
  <FilterInput filter={filter} onFilterChange={setFilter} />
  <LimitSelector limit={limit} onLimitChange={setLimit} />
  <SortSelector sortBy={sortBy} onSortChange={setSortBy} />
</div>
```

## ⚙️ Apply The Sorting Logic

It will show in the UI but we need to apply the sorting logic to the data. we can do this by updating the `filterCoins` function:

```
const filteredCoins = coins
  .filter(
    (coin) =>
      coin.name.toLowerCase().includes(filter.toLowerCase()) ||
      coin.symbol.toLowerCase().includes(filter.toLowerCase())
  )
  .slice() // 🔥 Important: make a shallow copy before sorting!
  .sort((a, b) => {
    switch (sortBy) {
      case 'market_cap_desc':
        return b.market_cap - a.market_cap;
      case 'price_desc':
        return b.current_price - a.current_price;
      case 'price_asc':
        return a.current_price - b.current_price;
      case 'change_desc':
        return b.price_change_percentage_24h - a.price_change_percentage_24h;
      case 'change_asc':
        return a.price_change_percentage_24h - b.price_change_percentage_24h;
      default:
        return 0;
    }
  });
```



We use `.slice()` to copy the array before sorting because the `sort()` method mutates the original array by default which we don't want. We want to avoid mutating state directly. This is a common pattern in React to ensure that we don't mutate state directly. `slice()` creates a shallow copy of the array, which just means that we create a new array that contains the same elements as the original array, but is a different object in memory. This is important because React relies on immutability to detect changes in state and re-render components accordingly.

We then use the `.sort()` method to sort the array based on the selected sorting option. We pass in a compare function with `a` and `b` as params.

These represent two items in the array that are being compared. The function returns:

- A negative number if `a` should come before `b`
- Zero if they're equal (no change in order)
- A positive number if `a` should come after `b`

This tells JavaScript how to reorder the array. For example, if we're sorting by price in descending order, we return `b.current_price - a.current_price` so that higher prices come first.

This compare function is run on every pair in the array to figure out the correct order.



## Summary

---

In this lesson, you learned how to fetch and manage API data in React using both `.then()` and `async/await`. You explored the `useEffect` hook for side effects, handled loading and error states, and built reusable components like `CoinCard` and `FilterInput`. By integrating the CoinGecko API, you practiced filtering, sorting, and displaying real-time data while understanding concepts like environment variables and React's Strict Mode. Overall, you gained the skills to confidently build dynamic, data-driven React applications.

Happy Coding! 💻✨





# Keep Learning. Keep Building.

## Thanks for reading this guide!

If this PDF helped you learn something new,  
consider following for more **high-quality developer resources**.

### You'll get:



React.js &  
Next.js Guides



JavaScript  
Deep Dives



Node.js &  
Express



MERN Stack  
Roadmaps



System  
Design



Interview  
Preparation



Real-world  
Projects



Free PDFs &  
Notes



### About the Creator

## Abhay Tiwari

MERN Stack Developer • Tech Content Creator

Helping developers learn practical skills  
through high-quality free resources.



### Follow for More

Stay updated with the latest  
guides, roadmaps, interview  
resources, and more.



### LinkedIn

[linkedin.com/in/abhay-tiwari-satna](https://linkedin.com/in/abhay-tiwari-satna)

Let's connect and grow together!



### More Resources

Follow for upcoming PDFs, roadmaps,  
interview questions, and developer  
resources.



If you found this guide useful,  
share it with another developer.  
It really helps! ❤️



© 2026 Abhay Tiwari

Happy Coding! ❤️